

Temă: Agent QA cu Tools + Prompts

Implementează tool-urile, prompts-urile și conectează-le într-un agent ReAct funcțional

Structura proiectului:

```
1  proiect/
2  |  agent.py                # QA agent / orchestrare LLM
3  |  |  tools/
4  |  |  |  __init__.py      # Exportă ToolWrapper
5  |  |  |  registry.py      # TOOL_REGISTRY + @register_tool
6  |  |  |  params_models.py # Pydantic BaseModel pentru fiecare tool
7  |  |  |  basic_tools.py   # calculator, get_datetime, web_search
8  |  |  |  tool_wrapper.py   # ToolWrapper.call() + ToolWrapper.catalog()
9  |  |  |  prompts/
10 |  |  |  |  registry.py    # PromptRegistry (load YAML + Jinja2)
11 |  |  |  |  planner.yaml
12 |  |  |  |  analyst.yaml
13 |  |  |  |  summary.yaml
14 |  |  |  |  extract.yaml
15 |  |  |  |  README.md
```

Ce trebuie făcut:

1

Tools cu Pydantic

- Pydantic BaseModel per tool (params)
- Decorator @register_tool
- ToolWrapper.call() + catalog()

2

Prompts cu YAML + Jinja2

- YAML: prompts
- PromptRegistry încarcă și render-uieste
- Variabile dinamice cu Jinja2

3

Agent QA - Chat with Tools

- agent.py invocă LLM cu tools + prompts
- Răspuns final clar pentru user

4

ReAct Pattern

- Loop Think → Act → Observe → Repeat
- max_iterations + error handling
- Multiple tool calls în același turn

Respectă structura • folosește convențiile din curs • testează cu un input real